
BuildDoctor

A GitHub App that reads failing CI logs, determines the cause, and comments on the commit responsible. A complete technical reference.

SERVICE	builddoctor.onrender.com
APPLICATION	github.com/apps/builddoctor-ci
SOURCE	github.com/aeturijayasimhayadav2004/builddoctor
SCALE	Approximately 9,600 lines across 14 development phases
DEPLOYMENT	Three live installations, one belonging to a third party
VERIFIED	28 August 2026, end to end against production

Every figure in this document was read from the running system or from source at the time of writing, rather than recalled.

Contents

1 **Summary**

What the system does, stated as it would be said aloud.

2 **Processing a failed build**

The ten stages from webhook receipt to published comment.

3 **Architecture**

Four components, their arrangement, and the reasoning behind it.

4 **Source inventory**

Every module, its size, and its responsibility.

5 **External dependencies**

Six third-party services, and the alternative rejected in each case.

6 **Data model**

Two tables, and the column governing access.

7 **Design decisions**

The choices requiring justification under examination.

8 **Examination questions**

Thirty questions with model answers and their purpose.

9 **Evidence of operation**

A production run recorded during preparation of this document.

NOTE ON USE

Sections 1 to 3 constitute an opening account of the system. Section 7 is the material distinguishing an engineer who built something from one who followed a tutorial: each entry is a decision, its justification, and the alternative rejected. Section 8 is for rehearsal.

Section 4 is *not* for memorisation. Its shape should be familiar; its detail should be looked up. An examiner tests whether a system can be explained, not whether an inventory can be recited.

1. Summary

1.1 Statement of purpose

BuildDoctor is a GitHub App that observes repositories for failing continuous-integration runs. When a build fails it downloads the job log, reduces it to the region surrounding the error, submits that extract to a language model together with the code change that triggered the run, and publishes a comment on the responsible commit explaining the failure.

It additionally classifies each failure into one of three lanes, and where a failure appears transient it re-runs the job without human intervention.

It remembers. Every diagnosis is stored with a vector embedding of the log, so that a recurrence of a similar failure supplies the earlier diagnosis to the model as context.

1.2 Classification lanes

TABLE 1.1 — LANES AND THEIR CONSEQUENT ACTIONS

Lane	Meaning	Action
informational	A genuine code or configuration error requiring human correction	Publish an explanation; take no further action
safe_auto_fix	Transient: network interruption, flaky test, runner fault	Re-run the failed job automatically
needs_review	Ambiguous, or affecting sensitive material	Mark for human attention

LIMITS OF AUTHORITY, TO BE STATED BEFORE BEING ASKED

The system **does not write code**. It does not push commits, open pull requests, or modify files, and the application's permissions do not permit it to do so: `contents` is granted at **read**. The only write operations available are publishing a comment, applying a label, and re-running a job.

This limitation is deliberate. An agent that edits source code on the strength of an inference is a liability rather than a feature.

1.3 Grounds for regarding this as production work

- **It has external users.** Three live installations, one belonging to a party unconnected with the author.

- **It is measured.** A 24-case reference set scores the classifier at **91.7 per cent** overall and **95.2 per cent** on unambiguous cases. Regressions are detected by re-running it.
- **It fails closed.** Every access decision defaults to refusal where evidence is absent.
- **It has survived a security incident.** A credential was exposed during development; the response is documented in the repository rather than concealed.

2. Processing a failed build

The following is the response to a request to describe the system's operation.

2.1 Receipt and authentication

1. **GitHub transmits a webhook.** A `workflow_run` event with action `completed` is delivered to `POST /webhook`. GitHub requires a response within **ten seconds**.
2. **The signature is verified.** HMAC-SHA256 is computed over the raw request body using the shared secret and compared using `hmac.compare_digest`. Failure returns 401 and nothing further executes. This precedes parsing, since parsing attacker-controlled input before authenticating it is itself the defect.
3. **The access gate is consulted.** The value of `installations.is_allowed` is read for the originating installation. Where the gate is closed the event is logged and discarded: no log is retrieved, no model invoked, and no expenditure incurred.
4. **A response is returned immediately.** Substantive work is deferred to a background task so that GitHub's ten-second limit is never at risk. A slow model response must not cause the application to be marked a failing integration.

2.2 Evidence gathering

5. **A scoped credential is obtained.** A JWT is signed with the application's private key and exchanged for an installation access token limited to that single installation and expiring after one hour. The token is cached in memory for its lifetime.
6. **Evidence is assembled.** Failed jobs are enumerated, their logs downloaded, and each reduced to **five lines preceding and ten following** every `##[error]` marker. The triggering change is retrieved, being either the pull-request diff or the commit against its parent.
7. **Memory is consulted.** The extract is embedded into 384 dimensions and a cosine search performed across previous diagnoses. A match exceeding **0.90** similarity is supplied to the model as context.

2.3 Determination and action

8. **The failure is classified.** The classification node invokes the language model with the extract, the diff, and any prior match, requesting a diagnosis and a lane as structured JSON.

9. **The result is routed.** The graph routes on the lane to an action node, which calls the tool server over HTTP to publish a comment, apply a label, or re-run the job.
10. **The outcome is recorded.** A row is written containing the repository, run identifier, lane, extract, diff summary, diagnosis text, installation identifier, and the embedding that renders this failure discoverable in future.

MEASURED, NOT ESTIMATED

End to end on a warm instance: approximately **eighteen seconds** from build failure to published comment. Recorded during preparation of this document; see section 9.

3. Architecture

3.1 Components

TABLE 3.1 — LOGICAL COMPONENTS AND THEIR DEPLOYMENT

Component	Responsibility	Deployment
Web application	Webhook receipt, sign-in, read API, administrative routes	FastAPI, single web service
Agent graph	Classification, routing, action dispatch	LangGraph, in-process
Tool server	The four operations that write to GitHub	Mounted at <code>/internal/mcp</code>
Dashboard	Diagnosis history; approval and revocation	React, served at <code>/dashboard/</code>

3.2 On the separation of the tool server

The obvious objection is that the tool server is not a separate service in production, and therefore that the separation is nominal.

The response is that **the boundary is the purpose, not the process count**. Every operation writing to GitHub is reachable by exactly one route: a described, typed tool invocation. The agent has no access to the GitHub client module. Consequently the complete set of actions this agent may perform upon a repository is four function signatures, all of which may be read in a minute.

Under Docker the components are genuinely separate containers. In production they were combined into one process, the free tier providing a single service. Purchasing a second service to render a diagram accurate, where the boundary is enforced in code regardless, would be a poor exchange.

3.3 The complete write surface

TABLE 3.2 — EVERY OPERATION AVAILABLE TO THE AGENT

Tool	Effect
<code>post_pr_comment</code>	Comment upon a pull request
<code>post_commit_comment</code>	Comment upon a commit
<code>add_pr_label</code>	Apply a label to a pull request

Tool	Effect
rerun_workflow_job	Re-run a failed job

3.4 The re-run loop guard

An agent capable of re-running failed jobs may loop indefinitely: the re-run fails, a webhook is delivered, a further re-run is issued. The guard employs GitHub's own attempt counter; only the first attempt is ever re-run, and a failing second attempt is classified but not re-run.

The property that matters is that this is **stateless**. The counter belongs to GitHub, not to a variable in memory that a restart would reset. That distinction is the difference between a guard and an expectation.

3.5 Exposure of the tool endpoint

The tool server is mounted within a public web service and its URL is therefore reachable. It refuses regardless: the `Host` header is compared against an allowlist and a mismatch returns **421 Misdirected Request**. A request arriving through the platform router carries the public hostname, which is not listed; the application's own call carries a loopback host, which is.

Both directions were verified prior to deployment: 421 for a foreign host, and a successful handshake enumerating all four tools over loopback.

4. Source inventory

4.1 Request path

Module	Lines	Responsibility
main.py	1311	All web routes, signature verification, the access gate, evidence gathering, and every installation-lifecycle handler. The automatic-approval rule resides here.
graph.py	332	The agent as a state machine: classify, route, act. Routing is a pure function of the lane, rendering control flow inspectable rather than emergent.
diagnose.py	521	Prompt construction, structured output, response validation, and fallback where the response is unusable.
log_excerpt.py	67	Token economy. A CI log may be megabytes; a context window is not. Small file, disproportionate effect upon cost and accuracy.
github_client.py	387	Every GitHub API call. Each function requires an installation identifier; there exists no ambient credential to employ inadvertently.
mcp_server.py	295	The four tools, and rebinding protection by host allowlist.
mcp_client.py	219	Invocation of those tools over HTTP; retry logic.

4.2 Memory

Module	Lines	Responsibility
embeddings.py	150	Reduction of a log extract to 384 dimensions using a local model: no API call, no per-embedding charge, and no second party receives the log.
memory.py	150	The similarity lookup. The threshold was derived by measurement; see section 7.
backfill_embeddings.py	110	Non-recurring: embedded rows written before memory existed, so that history was searchable rather than beginning at zero.
migrate_jsonl.py	139	Non-recurring: import of the original flat-file history into Postgres.

4.3 Identity, access and presentation

Module	Lines	Responsibility
app_auth.py	422	The application's own identity: signature, exchange for an installation token, caching.
user_auth.py	418	A person's identity: OAuth and the signed session cookie. The user's token is discarded and never stored.
db.py	976	Two tables, the migration runner, and every line of SQL in the project.
dashboard.py	325	Read-only API routes. No write occurs anywhere within it.
admin.py	221	Owner-only approval and revocation. Ownership is determined from GitHub rather than from configuration liable to drift.
notify.py	124	A single line of text to a chat webhook. Fire-and-forget; cannot raise, executing within a handler under time constraint.
server_combined.py	130	Combination of application and tool server into one process for deployment.
preflight.py	366	Interrogates all six external dependencies, the gate state of every installation, and the protected routes. To be run before demonstration.

4.4 Dashboard

Module	Lines	Responsibility
App.tsx	312	Layout and the three empty states: signed out, no installations, awaiting approval. Each is a distinct condition with a distinct remedy and is therefore worded distinctly.
api.ts	275	All requests. The API base defaults to same-origin, which is what permits the session cookie to function.
DiagnosisTable.tsx	244	The principal table, with filtering by lane.
AdminPanel.tsx	181	Approval and revocation, displayed only to the owner, indicating how each installation was approved.
RefreshAccess.tsx	67	Re-executes sign-in to refresh a stale installation list. A control rather than an automatic action, automation requiring the token to be retained.
Remaining components	~650	Stat cards, pending-approval notice, sign-in, loading skeletons, lane badges, the memory cell, icons and formatting.

4.5 Evaluation suite

Artefact	Purpose
<code>golden_set.json</code>	Twenty-four cases in four groups: synthetic, historical, middle-zone, and live cross-check. Each carries a known-correct lane and a written rationale.
<code>run_eval.py</code>	Executes the set against the production classifier , not a copy. It calls the real classification and memory functions but never routes to action nodes, and therefore cannot publish or re-run anything.
<code>findings.md</code>	The interpretation of the figures. Written by hand; survives re-execution.
<code>baseline_phase8.json</code>	A frozen run. Subsequent runs are compared against it, rendering "has this been made worse" an answerable question.

PRESENT SCORE

22 of 24, being 91.7 per cent overall; upon the 21 unambiguous cases, **20 of 21, being 95.2 per cent**. By lane: needs-review 9 of 9, informational 11 of 12, safe-auto-fix 2 of 3.

The three ambiguous cases are constructed to be arguable. Movement among them is noise; movement among the remaining 21 is signal. That distinction is the reason two figures are reported rather than one.

5. External dependencies

Six services the project does not own. In each case the material question is not what it is, but why that one.

5.1 GitHub, the platform

Source of webhooks, logs and diffs; destination for comments. Integrated as a **GitHub App** rather than by personal access token.

A personal access token carries one person's entire access to everything visible to them, does not expire until revoked, and cannot be scoped per repository. An App receives an installation token **limited to one installation and expiring within the hour unaided**.

When a credential was exposed during development, that distinction bounded the consequence.

5.2 Groq, model inference

Executes an open-weights twenty-billion-parameter model, called through the OpenAI SDK, the provider being API-compatible.

Groq was selected for latency and a free allowance; inference falls within GitHub's ten-second window, and although this is mitigated by deferring the work, latency remains material. An open-weights model was preferred to a frontier model because the task is bounded: read an error, name the cause, select one of three labels. The evaluation records 95.2 per cent upon unambiguous cases. Frontier pricing for that task would be waste.

The exchange is admitted: the SDK is OpenAI's, so a change of provider is a change of base URL. This is deliberate portability rather than dependence.

5.3 Neon, Postgres with pgvector

Stores every diagnosis and the installations table.

The hosting platform's own free database **expires thirty days after creation**. For most projects this is an inconvenience; here it is fatal, the purpose of the embedding search being recall of failures from weeks past. A database that deletes itself monthly is not memory. Neon's free tier does not expire.

pgvector was preferred to a dedicated vector database because at this scale the vectors belong beside the rows they describe. A separate store would introduce a service, a

synchronisation problem, and a second failure mode, in order to solve what an ordinary ordered query already solves.

5.4 Render, hosting

One free web service running the application and tool server, deploying automatically upon push.

The cost of the free tier is that the instance sleeps after approximately fifteen minutes idle and requires one to two minutes to wake. **Webhooks arriving during sleep are lost outright**, GitHub not retrying. This was measured rather than assumed.

5.5 An external scheduler, availability maintenance

Requests the health endpoint on a schedule so that the instance does not sleep and no webhook is lost.

GitHub Actions' own scheduler was attempted first and **does not work**. Measured across seven hours, a ten-minute schedule should have fired approximately forty-two times and fired **none**. Scheduled workflows are deprioritised under load without error or notification. An external requester fires punctually.

5.6 A chat webhook, owner notification

One line of text when a private-repository installation requires manual approval.

Presently unconfigured by choice; the message is written to the server log instead. The consequence is stated plainly in the installation guide: an installer must contact the owner, nothing else having done so.

5.7 Library dependencies

TABLE 5.1 — EACH LIBRARY AND THE FUNCTION IT IS LOAD-BEARING FOR

Package	Purpose
fastapi, uvicorn	Asynchronous web framework. Asynchrony is material: a handler awaiting a model response must not block others.
langgraph	The agent as an explicit state machine, preferred to a prompt-chaining loop because routing becomes inspectable and testable.
mcp	The tool boundary, rendering the write surface enumerable.
sentence-transformers	Local embeddings: no charge, no third party, no network call in the critical path.

Package	Purpose
pgvector, psycopg, sqlalchemy	Vector column, Postgres driver, schema and queries.
pyjwt[crypto]	Signs the application credential. Preferred to a GitHub SDK, the flow being two API calls and a signature; an SDK would be a dependency concealing nine lines.
itsdangerous	Signs the session cookie.
openai, httpx	Client for the compatible inference API; asynchronous HTTP throughout.

6. Data model

Two tables, deliberately few.

6.1 The diagnoses table

TABLE 6.1 — RECORD OF EACH FAILURE PROCESSED

Column	Content
id	Primary key
run_id	The workflow run identifier, linking to the originating run
repo	Owner and name
created_at	Timestamp
log_excerpt	The reduced log: precisely what the model received
diff_summary	The change that triggered the run
diagnosis_text	The model's explanation
posted_to	Destination of the published comment
lane	The classification assigned
embedding	A 384-dimension vector, the subject of the memory search
raw	The complete model response, retained for diagnosis
installation_id	The originating installation: the multi-tenant key

6.2 The installations table

TABLE 6.2 — RECORD OF WHO MAY USE THE SYSTEM

Column	Content
installation_id	The identifier for the installation
account_login	User or organisation name
account_type	User or organisation
is_allowed	The gate , consulted upon every webhook
approval_source	Automatic, owner, or null

6.3 The necessity of recording the approval source

Installations covering only public repositories approve themselves. An installation formerly all-public may subsequently acquire a private repository, at which point automatic approval represents a promise no longer true and must be withdrawn.

The difficulty is that an allowed installation appears identical whether set by the application or by a person. Withdrawing indiscriminately would permit the application to overrule a decision the owner made deliberately. The reason is therefore recorded, and only automatic approvals are ever withdrawn automatically.

Rows predating the column are **null rather than backfilled to owner-approved**, backfilling being the writing of a supposition into a column that governs access.

6.4 Withdrawal is unidirectional

Making a repository private withdraws approval. Making it public again does **not** restore it.

Were restoration automatic, approval would be reversible by any party able to alter a repository's visibility, placing the gate in the hands of the applicant rather than the approver. This was demonstrated in production: the privatisation event closed the gate, and the publication event arriving three minutes later produced no effect.

6.5 Schema evolution without a migration framework

Migrations are a list of description and statement pairs executed at startup, each written to be idempotent so that repetition is safe. A migration framework was not adopted because, at two tables and a small number of additive changes, it constitutes a dependency, a directory and a conceptual model addressing a problem the project does not yet have. The occasion for adopting one is the first destructive migration, not the passage of time.

7. Design decisions

Each entry states a decision, its justification, and the alternative rejected.

7.1 The similarity threshold was derived rather than assumed

Memory assists only if similarity is meaningful. The threshold was established by measuring embedding distances between known-related and known-unrelated failure pairs and selecting the value separating them. Below it, unrelated failures matched and the supplied context made diagnoses worse.

The evaluation subsequently identified a defect in the untested region near the boundary. That finding, and its correction, are documented. A figure defensible by measurement is superior to a figure that merely happens to work.

7.2 Failing closed throughout

- A repository object lacking the visibility key is treated as **private**, so that a change of payload shape causes installations to await a human rather than to proceed.
- An empty repository list constitutes **absence of evidence**, and absence of evidence cannot support assent.
- An installation covering all repositories is refused **even where all are presently public**, since it silently extends to repositories created subsequently.
- A model response that cannot be parsed falls back to the lane whose only action is to publish a comment.

7.3 The payload field was confirmed rather than assumed

The webhook reference documents that an installation payload contains a repositories array, but **not the contents of each object**. Assuming the field name returned by the REST API would have shipped a defect silently causing every installation to appear private.

Instead a real delivery was retrieved from GitHub's own delivery log and the objects confirmed to carry exactly five keys. **Interrogate the system that owns the fact.** This habit is the most transferable element of the project.

7.4 Permission changes are asymmetric

Before narrowing the contents permission, the documentation was consulted rather than assumed. Adding permissions requires every installation to accept anew; removing them takes effect immediately. This was subsequently confirmed in practice: both installations

moved to read access with identical modification timestamps, without suspension and without action by anybody.

7.5 The user's credential is discarded

At the conclusion of the sign-in exchange the access token and its refresh token are **discarded**. Only the user identifier and login are placed in a signed session cookie.

The cost is stated: the installation list is a snapshot taken at sign-in, so a newly installed application is invisible until sign-in is repeated, which is the purpose of the refresh control. The alternative rejected was retention of a live credential in a signed but unencrypted cookie, or in the same database the dashboard reads.

7.6 A credential was exposed, and the response is documented

An exposed secret was discovered during development. It was rotated, the exposure confirmed closed, and the incident recorded in the repository rather than quietly corrected. The reason the consequence was contained is the first decision in section 5: scoped, self-expiring installation tokens rather than a personal access token.

7.7 Knowing what has not been built

The repository maintains a record of deferred work: no pagination, no second approver, no refresh-token policy, no scheduled visibility re-check. Each states the condition that would occasion building it. The occasion is the row count, not the calendar.

The capacity to enumerate one's own omissions, together with the condition that would alter the judgement, reads as engineering discipline rather than as an unfinished project.

8. Examination questions

Thirty questions. The closing italic states what each is intended to establish.

8.1 Opening

Q1 Describe this project.

Employ the statement of purpose in section 1. Thirty seconds, then stop and permit the examiner to direct.

ESTABLISHES capacity for compression. Discursiveness here is expensive.

Q2 Describe what occurs when a build fails.

The ten stages of section 2. Emphasise the order: signature, then gate, then response, then work. The response precedes the work because of the ten-second limit.

ESTABLISHES whether the control flow is understood or was arrived at by trial.

Q3 Why an agent? Could this not be a script?

Candidly: most of it could. The agent earns its place at a single point, namely the determination of which action to take. Classification and routing form a state machine because the lane determines the action, and that must be inspectable and testable rather than buried within prompt text. Were it only ever to publish comments, a script would suffice.

ESTABLISHES whether the term is used as description or as fashion. Conceding the limit is the strong answer.

Q4 What was the most difficult part?

The case of a repository added to an existing installation. Automatic approval asserts that every covered repository is public, yet repositories may be added afterwards. Withdrawing indiscriminately would overrule a deliberate human approval; not withdrawing would extend automatic approval into private repositories. Resolved by recording why the row was approved and withdrawing only the application's own decisions.

ESTABLISHES whether a genuinely difficult design problem was encountered, as distinct from implementation friction.

8.2 Architecture

Q5 **Why a tool protocol rather than calling the API directly?**

To render the write surface enumerable. Every operation touching GitHub passes through one described tool invocation, so the complete set of permitted actions is four signatures. Without that boundary, the question of what the agent may do to a repository is answered only by reading the entire codebase.

ESTABLISHES *whether a technology was chosen or a fashion followed.*

Q6 **Why a graph framework rather than a plain loop?**

Explicit state and explicit routing. The lane determines the subsequent node by a pure function, so control flow is a graph that may be read. A prompt-chaining loop conceals routing within the model's output, which is precisely the part most requiring test.

ESTABLISHES *framework literacy, and whether a trade-off can be articulated rather than a preference.*

Q7 **Your tool server runs in the same process. Is the separation not illusory?**

A fair objection, and the process boundary is indeed illusory; the code boundary is not. Under Docker the components are separate containers. On a single free service they were combined. The enforcement that matters, namely that the agent reaches GitHub only through described tools, is unchanged. Purchasing a second service to render a diagram accurate would be the wrong exchange.

ESTABLISHES *whether a design is defended under pressure or abandoned. Concede the real point; retain the substantive one.*

Q8 **How is indefinite re-running prevented?**

By GitHub's own attempt counter. Only the first attempt is ever re-run. Because the counter belongs to GitHub rather than to process state, neither a restart nor a redeployment can reset it.

ESTABLISHES *whether the superiority of stateless guards is understood.*

Q9 **What occurs if the model returns nonsense?**

Structured output is requested and validated. An unparseable or invalid response falls back to the informational lane, whose only action is to publish a comment. The failure mode is a less useful comment, never an unintended write.

ESTABLISHES *whether the design anticipates the model being wrong.*

Q10 How would this scale to a thousand repositories?

Three things fail first, in order. The free instance, webhook processing being in-process background tasks, requiring a genuine queue. The diagnoses route, which employs a limit rather than pagination and silently truncates. The vector search, which is unindexed and requires an index, being one statement.

ESTABLISHES *whether one's own bottlenecks can be identified. Three, in order, is materially better than a generality.*

8.3 Security and access

Q11 How do you authenticate to GitHub?

As a GitHub App. A credential is signed with the application's private key and exchanged for an installation access token limited to one installation and expiring within the hour, cached in memory for its lifetime. No personal access token exists anywhere in the system.

ESTABLISHES *whether the implemented flow is genuinely understood.*

Q12 Why not a personal access token, which is simpler?

Such a token carries one person's entire access, does not expire until revoked, and cannot be scoped per repository. An installation token is scoped and self-expiring. When a credential was exposed, that distinction bounded the damage.

ESTABLISHES *reasoning about blast radius rather than recitation.*

Q13 How is a webhook verified as genuine?

By a keyed hash computed over the raw request body using the shared secret and compared in constant time, so that the signature is not disclosed through timing. The check precedes parsing, since parsing attacker-controlled input before authenticating it is the defect.

ESTABLISHES *whether the reasons for constant-time comparison and for ordering are known.*

Q14 Who may approve an installation?

Only the account that registered the application, determined from GitHub at the time of the request rather than from configuration liable to diverge from reality.

ESTABLISHES *the authoritative-source habit a second time.*

Q15 Automatic approval of public repositories: is that not a vulnerability?

It is not, and the reasoning is the whole argument. A public repository's build logs are already readable by anyone. Reading one discloses nothing previously closed, so requiring approval would be permitting what is already permitted. Private logs are a genuine decision and a person still makes it. The rule is narrow, requiring both an explicitly selected repository list and every repository public.

ESTABLISHES *whether one can reason about what a control protects, rather than adding controls reflexively.*

Q16 Why refuse an all-repositories installation if everything is public today?

Because approval is a statement about the future. Such an installation covers every repository the account subsequently creates, and the available evidence on whether the relevant event is delivered for those repositories is self-contradictory. The assertion therefore cannot be honoured and is refused rather than guessed.

ESTABLISHES *whether ambiguous evidence is met by failing closed.*

Q17 A repository is made private after automatic approval. What then?

The corresponding event is delivered and approval withdrawn, but only where the source was automatic. An owner-approved installation is never withdrawn automatically, that being the application overruling a deliberate human decision.

ESTABLISHES *whether the design extends beyond the expected path.*

Q18 It is made public again. Is approval restored?

Deliberately not. Were it restored, approval would be reversible by any party able to alter a repository's visibility, placing the gate in the hands of the applicant rather than the approver.

ESTABLISHES *whether a privilege-escalation path can be identified in one's own design.*

Q19 Where is the user's token stored?

Nowhere. It is discarded at the conclusion of the sign-in exchange together with the refresh token. The session cookie holds the user identifier and login, signed.

ESTABLISHES *session design, and awareness of what is being traded.*

Q20 The administrative endpoint has no request-forgery token. Why is that acceptable?

A fair challenge. The mitigation is that the route requires both a valid session and that the caller be the owner, and it was verified that an unauthenticated request with a well-formed body returns 401 with the row unchanged. It is not a substitute for protection upon a larger surface; with one write route and one authorised person it is a stated and bounded gap rather than an oversight.

ESTABLISHES the handling of a genuine weakness. Defending it as flawless is the wrong answer; stating the bound is the right one.

Q21 The tool server is on a public URL. Is it exposed?

Reachable, not usable. The host header is compared against an allowlist and a mismatch returns 421. Traffic through the platform router carries the public hostname, which is not listed; the application's own loopback call carries a host that is. Tested in both directions.

ESTABLISHES awareness of rebinding attacks, which is a genuinely advanced answer.

8.4 Data, memory and evaluation

Q22 Explain the memory feature.

Each log extract is embedded into 384 dimensions by a local model and stored in a vector column. Upon a new failure a cosine search locates the nearest previous diagnosis, and above the threshold it is supplied to the model as context. It is retrieval-augmented generation across the project's own history.

ESTABLISHES whether embeddings can be explained without vagueness.

Q23 Where did the threshold come from?

Measurement. Distances were compared between known-related and known-unrelated pairs and the threshold set where they separated. Below it, unrelated failures matched and the supplied context degraded the diagnosis. The evaluation later identified a defect in the untested region near the boundary, which was corrected and documented.

ESTABLISHES — and this is the single question most worth preparing — whether a number was measured or felt. "It seemed right" fails.

Q24 Why a local embedding model rather than an API?

Cost, latency and disclosure. It is free per call, introduces no network hop into the critical path, and does not transmit the log to a second third party. The weights are contained within the image so that a cold start does not download them.

ESTABLISHES *practical reasoning about cost and latency.*

Q25 How is the classifier known to be adequate?

By a 24-case reference set with known-correct lanes in four groups, including a deliberately ambiguous middle zone. Presently 91.7 per cent overall and 95.2 per cent upon unambiguous cases. It exercises the production classifier but never routes to action nodes, so execution publishes nothing.

ESTABLISHES *whether evaluation occurs at all. Most comparable projects have none, and this answer is conspicuous.*

Q26 Why report two accuracy figures?

Because three of the cases are constructed to be arguable. Movement among them is noise; movement among the remainder is signal. A single blended figure would conceal the distinction rendering the metric useful.

ESTABLISHES *literacy in metrics, which is a strong differentiator.*

Q27 Why Postgres with a vector extension rather than a vector database?

The vectors describe rows already residing in Postgres. A dedicated store would introduce a service, a synchronisation problem and a second failure mode, to solve what one ordered query already solves. The occasion to revisit is scale, and the first remedy would be an index rather than a new service.

ESTABLISHES *resistance to architecture chosen for its appearance on a résumé.*

8.5 Operation

Q28 What happens if the server is asleep when a webhook arrives?

It is lost; GitHub does not retry. This was measured rather than assumed. The remedy is an external requester maintaining availability. The platform's own scheduler was attempted first and does not work: across seven hours a ten-minute schedule should have fired approximately forty-two times and fired none, silently.

ESTABLISHES *knowledge of one's real failure modes. Admitting lost deliveries is far stronger than claiming unprovable reliability.*

Q29 How is it known to be working at present?

By a preflight script performing ten checks in approximately thirty seconds. It interrogates each dependency directly, verifies the gate state of every installation, confirms the deployed bundle, examines recent delivery outcomes, and asserts that the protected routes still refuse anonymous callers. It exists because free-tier dependencies fail silently and availability proves nothing about the others.

ESTABLISHES *operational maturity. Very few comparable projects possess this.*

Q30 What would you do next?

In order: a genuine job queue, so that processing survives a restart; pagination, before the row count renders the present limit dishonest; a scheduled visibility re-check, withdrawal presently depending upon deliveries that a sleeping server loses; and an index when the search becomes slow.

Each has a stated occasion. None is a rewrite.

ESTABLISHES *prioritisation. Occasions rather than aspirations are the distinction.*

9. Evidence of operation

Recorded 28 August 2026. Not an account of intended behaviour but a record of observed behaviour.

9.1 The recorded run

```
16:01:09    commit 78e7707 pushed to builddoctor-testing/ci-test
16:01:2x    workflow run 33187951570 concluded: failure
16:01:38    comment published by builddoctor-ci[bot]
16:01:39    diagnoses row id=12 written, embedding stored

elapsed:    approximately 18 seconds, failure to published comment
```

9.2 The published comment

```
BuildDoctor: CI failed

The job failed because a step in the workflow ran the command
`exit 1`, which immediately caused the job to terminate with exit
code 1. The added `trigger.txt` file is unrelated and does not
explain the failure.

Failed job(s): build
Lane: informational - STEP 2 (build or dependency machinery):
The failure originates from a workflow step (`Run exit 1`)
defined in the CI configuration, not from application code.
```

WHY THIS OUTPUT MERITS PRESENTATION

The system did not merely read the error; it **excluded the diff**. The commit added an unrelated file, and the model stated explicitly that the file does not explain the failure. That is the evidence-gathering stage performing work a log-only tool could not perform.

9.3 Preflight, same session

```
[ ok ] environment: all 8 keys present
[ ok ] render: not suspended; deploy live
[ ok ] server: awake, /health in 0.58s
[ ok ] dashboard: serving bundle index-0tVG-ofB.js
[ ok ] database: 3 installs, all open
[ ok ] groq: key valid, model available
[ ok ] github app: bullddoctor-ci, 3 installs, perms correct
[ ok ] webhooks: last 20 delivered ok
[ ok ] auth gate: all 3 routes return 401
[ ok ] git: clean and pushed

10 pass, 0 warn, 0 fail
```

9.4 Multi-tenant traffic

TABLE 9.1 — THE FOUR MOST RECENT DIAGNOSES

Row	Repository	Installation	Lane
12	bullddoctor-testing/ci-test	157254438	informational
11	Dhatubablu/samplerepofirst	157269524	informational
10	Dhatubablu/samplerepofirst	157269524	informational
9	bullddoctor-testing/ci-test	157254438	informational

Rows 10 and 11 originate from an installation unconnected with the author. Multi-tenancy is therefore demonstrated by traffic rather than asserted by design.

IF ONE THING IS TO BE DEMONSTRATED

Open the commit and display the published comment; then open the dashboard and display the same event as a row. Two artefacts, one event, thirty seconds. That sequence establishes the entire pipeline without recourse to slides.